FloSports Event Explorer PRD (Staff Full Stack Engineer)

FloSports Event Explorer

PRD: Event Listing with Filterable UI

Context

FloSports covers over 200 properties across 25+ sports — from wrestling and track & field to cycling and cheerleading. Our internal tools team needs a simple way for operations staff to browse and filter upcoming, live, and completed events across the platform.

Your task is to build a proof-of-concept: a NestJS backend that aggregates event data from two separate sources, and an Angular frontend with a filter bar component that queries the backend.

The Data

This project uses two separate data sources — simulating a BFF that aggregates from multiple upstream services.

Source 1: Event Catalog (`events.json`)

The core event listing. Each event looks like this:

```
{
  "id": "evt-0001",
  "title": "NAIA Outdoor Track & Field Championships - Day 1 Finals",
  "sport": "Track & Field",
  "league": "NAIA",
  "status": "upcoming",
  "startTime": "2026-02-21T12:15:00.000Z"
}
```

Source 2: Live Stats (`live-stats.json`)

Streaming statistics for currently live events. Each record looks like this:

```
{
  "eventId": "evt-0003",
  "viewerCount": 4171,
  "peakViewerCount": 4955,
  "streamHealth": "good",
  "lastUpdated": "2026-02-17T23:07:47.146Z"
}
```

Field notes:

- `status` is one of: `upcoming` , `live` , `completed`
- Live stats exist only for events where `status` is `live`
- `streamHealth` is one of: `excellent` , `good` , `fair` , `poor`

Download the data:

- events.json
- live-stats.json

Your API responses should merge data from both sources — a single event in the API response should include both its catalog data and its live stats (when available). How you load, store, and merge this data is up to you. No database

is required, but you're free to use one if you prefer.

Requirements

Backend (NestJS)

Build a REST API that aggregates from both data sources and serves the frontend's filtering needs. The API should allow the frontend to:

- Retrieve events with support for filtering by sport, status, and title search
- Get the list of available sports in the dataset
- Look up a single event by its ID
- Include live stats (viewer count, peak viewers, stream health) for live events in the response

Design the API however you see fit.

Frontend (Angular)

Build a single page that renders a **filter bar component** with three filters and displays the filtered results below it.

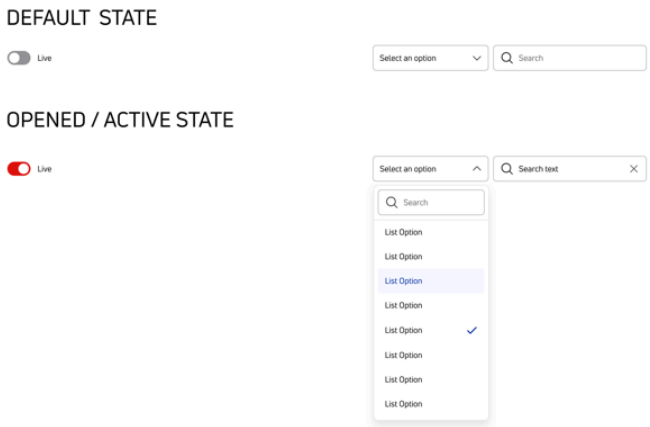
Design note: the filter bar needs to be an exact match with the design. The results section can be minimal, this is to verify the data connection works — we're evaluating the filter bar.

The three filters:

1. **Toggle** — "Live only" on/off switch that filters to `status: live`.
2. **Search** — Text input that filters events by title.
3. **Dropdown** — Single-select dropdown with search that lets the user filter by sport.

Filtering should happen server-side via the API, not client-side.

Important: Do not use any third-party UI component libraries (e.g., Angular Material, PrimeNG, ng-bootstrap, etc.). All components — including the toggle, search input, and dropdown — should be built from scratch. We want to see how you approach component architecture, accessibility, and styling on your own.



Technical Expectations

- **Backend:** NestJS with TypeScript

- **Frontend:** Angular (latest stable) with standalone components. Use SCSS for styling.
- **Single repo** — Keep both the backend and frontend in one repository. Use Nx, npm/yarn workspaces, or a simple root `package.json` with `concurrently`. We should be able to clone one repo and start both servers with a single command.
- **Testing:** Write tests for what matters most. We care about what you chose to test and why.

Your README

This matters as much as the code. Please address:

1. **Assumptions you made** — What wasn't explicit in this PRD that you had to decide on your own?
2. **API design** — How did you design the API? What are the endpoints, and why did you structure them that way?
3. **Data loading & merging** — How do you load the two data sources? How and where does the merge happen? What would change if these were real upstream HTTP services instead of files?
4. **Backend decisions** — How did you structure the NestJS service? How does the filtering work?
5. **Trade-offs** — What did you prioritize and what did you skip? What would you add with another 2 hours?
6. **AI tools** — Did you use AI assistants? What for? Was there anything they got wrong that you had to fix?

How to Work

Work on this the way you'd normally work on a real task. Use whatever tools are part of your usual workflow — AI assistants, documentation, Stack Overflow, whatever you reach for day-to-day. We're not testing whether you can code in a vacuum; we're looking at how you build things.

- **Commit as you go** — we'd like to see how you approached the problem over time.

Time Expectation

This should take approximately **2 hours**. We genuinely mean that — please don't spend a full weekend on this.

If you run out of time, write about what you would have done next. We'd rather read about your thinking than see half-finished features.

Submission

Create a GitHub repo with your solution and share the link with us. Please make sure the repo is public or that you've granted access to the email addresses we provide.

Include a screenshot / recording of your working application when you submit the take home assignment.

Deadline: 72 hours from when you receive this PRD.

What happens next: If we move forward, the next step is a live session where you'll walk through your code with one of our engineers. Come ready to talk about your decisions — especially around the filter architecture and how the frontend and backend work together.